· DDice ·

A Chronicle of Commands · Game Master's Edition

— Contents —

Game Master Commands

All commands in this chapter require the GM role.

Game Master Rolls

`gmr 1d20+5`	*(public roll in channel)*
`gmr 1d20+5 perception`	*(with label)*
`gmrs 1d20+5`	*(secret — sent to Game Master DMs only)*
`gmrs 1d20+5 stealth`	

Game Master HP & Rerolls Targeting

```
!hp @user +5    !hp @user -3
!hp +5 @user    !hp -3 @user
!rerolls @user +1    !rerolls @user -1
```

Game Master Rest Targeting

```
lrest @user   srest @user   hpfull @user   hphalf
@user
```

Preset Tags

```
/tag assign user:@player tag:Hero of Kalidale
/tag remove user:@player tag:Hero of Kalidale
/tag list user:@player
```

Custom Tags

```
/tag custom action:Create emoji:[any emoji]
name:MyTag
/tag custom action:Delete name:MyTag
/tag custom action:List
```

For server emojis: type \:emojiname: in chat to get the full ID.

`/char set field:str value:14 user:@player`	*(Game Master only)*

Items & Pages

`/char give user:@a item:A tarnished silver key \ note:Cold to the touch`	*(Game Master only)*
`/char take user:@a id:3`	*(by the number in their inventory)*
`/char edit user:@a id:3 item:Silver key note:Cold`	*(reword an item)*
`/char summary user:@a`	*(their sheet and every page at once)*
`/char standing user:@a`	*(merit and renown, and where each came from)*
`/char rollhistory user:@a`	*(every natural die they have rolled)*

*Items are free text — whatever a story calls for. Activities can hand them over too, and a player reads them back with **/char inventory**. Lore submitted with **/char lore** arrives in the sheet approval channel with Approve / Reject buttons, and a rejection asks you why.*

Sheet Import

Paste an exported sheet into any channel the bot watches.

`[paste sheet] @player`	*(Game Master importing for another player)*

Config & Maintenance

Config (Admin only)

`/config gmrole role:@Role`	*(add a GM role — several can be set)*
`/config gmrole role:@Role remove:true`	*(remove one)*
`/config gmrole role:@Role replace:true`	*(make it the only GM role)*
`/config gmrole`	*(list current GM roles)*
`/config heal charges 3`	
`/config statallowance points:15 minimum:1`	*(player build points (omit both to view))*
`/config hpbase base:3`	*(max HP = CON + this (default 2))*
`/config autorest action:List`	*(every recovery schedule)*
`/config autorest action:Add or update name:Breather \ hours:6 hp:50% rerolls:0% heal:0%`	*(a light top-up)*
`/config autorest action:Run now name:Breather`	*(fire one immediately)*
`/config npcchannel #channel`	*(set the NPC image bank channel)*
`/config npcreroll threshold:8`	*(NPC auto-reroll on nat ≤ N — 0 disables)*
`/config fightping enabled:true`	*(@-mention players on their turn — off by default)*
`/config rollaudit channel:#gm-rolls`	*(mirror every roll — raw input, result and jump link)*
`/config rollaudit test:true`	*(send a test mirror, report problems)*
`/config npcstats enabled:true`	*(reveal NPC stat blocks — hidden by default)*
`/config approvals channel:#sheet-approvals`	*(new sheets need GM sign-off)*
`/config approvals list:true`	*(every sheet still waiting — from the database)*
`/config approvals disable:true`	*(turn approval off)*
`/config rest type:Short Rest hp:50% rerolls:0%`	*(% of max)*
`/config rest type:Short Rest hp:3 rerolls:1`	*(flat numbers)*
`/config cleanwebhooks`	*(remove orphaned NPC webhooks)*

NPC roll cards hide STR/CON/DEX/WIS/LCK, the roll modifier and HP totals by default. Players see the name, order, the final roll total, the exact damage each hit deals, and a condition (unhurt / wounded / badly hurt / near death / down) instead of numbers — so the fight reads clearly

*without revealing what an NPC can take. Reveal everything with **/config npcstats enabled:true**. NPC management commands are Game Master-only regardless.*

*A sheet that is **still waiting** can be edited by its owner — spot a mistake before a Game Master gets to it and you can fix it, which retires the old request and posts a fresh one. Only an **approved** sheet is frozen to its owner.*

With an approval channel set, a player's new sheet is posted there for sign-off, pinging every GM role, with Approve / Reject buttons. Until approved they can't roll, heal or take fight actions — and once approved, their whole sheet can only be changed by a GM. Sheets a GM creates or edits skip the queue, and sheets made before approval was enabled keep working until someone edits them.

*Every way a player can write to their own sheet goes to the queue: **/char create**, **/char set** (any field — stats, order, class and weapons alike), **/char weaponemoji**, **/profile load** and pasting an exported sheet. Building a character one **/char set** at a time is not a way past a GM. Editing a sheet that is still pending retires the old request and posts a fresh one, so the channel holds one live entry per player rather than a pile of stale ones. A rejected sheet can still be edited by its owner — that is how they fix it — and each edit sends it straight back for another look.*

*Pressing **Reject** opens a box asking **why**. The note is optional, but it travels with the decision — it is written onto the request in the approval channel, sent to the player with the rejection, and shown again if they try to roll before fixing it, so nobody is left guessing at what to change. Declining an **export** asks the same way.*

*A **rejection is never the end of it.** A rejected sheet stays editable by its owner, so the player fixes whatever the GM objected to with **/char set** or **/char create** and rejoins the queue on their own. If they believe it was right as it stood, **/char submit** sends it again unchanged. There is no limit — a sheet can go back and forth as many times as it takes, and each resubmission retires the previous request so the channel still holds one live entry per player. The rejection notice spells out both routes, so nobody is left waiting on a GM to do it for them.*

*A declined **export** works the same way — running **/char export** again puts a fresh request in front of the GMs.*

*Sheets are accepted from **any channel the bot can read** — an ordinary text channel, a thread, a forum post, an announcement channel, or the text chat inside a **voice or stage channel**. Wherever it came from is recorded with the request, so the decision notice can find its way back there if the player's DMs are closed.*

*The queue lives in the database, not in a Discord message. **/config approvals list:true** shows every sheet still waiting — who, how long ago, and which channel it came from — even if the request was never posted, was deleted, or landed somewhere nobody reads. If the bot cannot post to the approval channel it says so on that list and pings the GM roles in the channel the sheet was submitted from, so a player is never left locked out in silence.*

***The roll-audit mirror records every roll, in every channel, with nothing skipped.** Prefix rolls, stat shorthand, success checks, rerolls, heals, **/roll**, **/dr** and every fight roll — plus Game Master rolls, including secret **gmrs** ones, so Game Masters are accountable to one another.*

*A Game Master rolling as an NPC with **/pr roll** is logged under their own name, tagged with the NPC they spoke as — the roll itself goes out through the NPC's webhook, so the audit is the only place it ties back to a person.*

*Rolls the bot makes for itself are recorded too, attributed to the fighter and tagged **auto**: auto-pilot attacks, defences and reroll answers, initiative at the start of every fight and whenever an NPC joins mid-fight, every roll of a full **/fight auto**, and demo bouts. Rolls typed inside the audit channel itself are mirrored as well — a secret **gmrs** there goes to the Game Master's DMs, so without the mirror it would leave no record at all.*

*Use **/config rollaudit test:true** to check it works. Set the channel's Discord permissions so only Game Masters can view it.*

*Several GM roles can be set at once — holding any of them grants GM access. Anyone with the Discord **Manage Server** permission always counts as a GM, so you can never lock yourself out by mis-setting a role.*

*Rest amounts come in two shapes, and the difference matters. A bare value **sets** the resource: **50%** puts them on half their maximum, **4** puts them on exactly 4 — even if that is fewer than they had. Prefix a **+** and it **adds** instead: **+4** gives four more HP, **+25%** gives a quarter of their maximum on top of what they have. Both cap at the maximum and both round down. **0%** leaves that resource untouched, and only the values you provide change.*

Scheduled Recovery

A server can run **any number of named schedules**, each with its own timing and its own strength. A light top-up every few hours and a full recovery overnight can sit side by side.

`/config autorest action:Add or update name:Breather \ hours:6 hp:50% rerolls:0% heal:0%`	*(half HP, rounded down, every 6h)*
`/config autorest action:Add or update name:Full Recovery \ hours:24 hp:100% rerolls:100% heal:100%`	*(everything back, once a day)*
`/config autorest action:List`	*(what is set, and when each next falls)*
`/config autorest action:Run now name:Breather`	*(fire one immediately)*
`/config autorest action:Pause name:Breather`	*(stop it without deleting it)*
`/config autorest action:Remove name:Breather`	*(delete it)*

Amounts use the same tokens as **/config rest**. A bare value **sets** the resource — **100%** full, **50%** half, **4** exactly four. A **+** prefix **adds** instead: **+4** is four more HP, **+25%** a quarter of their maximum on top. **0%** leaves it alone. Everything caps at the maximum and rounds down, so half of 11 HP is 5. A typo is refused when you set the schedule rather than quietly doing nothing at three in the morning.

Anyone on a quest that is in progress is skipped by every schedule. Their HP, rerolls and charges stay exactly where they are until the quest is completed, so being out in the field costs something. Applicants, and members of quests still open or already finished, are restored as normal. Heal charges only go to White Knights with WIS 5+, as everywhere else.

Give a schedule a **channel:** and each run is announced there, naming what it did, who was restored and who was left out in the field.

Each schedule carries its own clock in the database, so a restart or redeploy can neither skip a cycle nor fire one early. Adding a schedule, or resuming a paused one, starts its count from that moment.

Help & Maintenance

`/help`	*(overview of all command groups)*
`/help category:dice`	*(detail on a specific group)*
`/lastroll`	*(recall your last roll in this channel)*
`/backup now`	*(take one immediately — Game Master)*
`/backup auto channel:#backups`	*(automatic backups — Game Master)*
`/backup auto channel:#backups hours:12`	*(or a different interval — Game Master)*
`/backup auto channel:off`	*(stop them — Game Master)*

Backups

With a channel set, the database is posted there **every 24 hours** — and each backup **deletes and replaces the last**, so the channel holds exactly one file: the newest. **/backup now** takes one on demand and replaces the standing post the same way; with no channel set it comes back to you privately as a one-off copy instead.

*The cycle is measured from the **last backup taken**, not from when the bot last started. A redeploy mid-cycle resumes where it left off, and one that happens while a backup is overdue takes it shortly after boot — so a server that redeploys several times a day still gets its daily file. Switching backups on takes one immediately rather than waiting out the first cycle.*

*The file is a **settled snapshot**, not the live database — taken with SQLite's own **VACUUM INTO**, so a write landing mid-upload cannot produce a torn file, and unused pages are dropped so the attachment is as small as it can be. The new backup is posted **before** the old one is removed, so a failure part-way leaves the previous file in place rather than none at all.*

Destructive actions (/npc delete, /fight end, /quest delete, /weapon remove) ask for Confirm / Cancel before running.

What the Stats Do

`/stat`	*(what each stat is for, in plain words)*

The Server Weapon List

Weapons players can pick from are kept as a server list, so names stay consistent.

`/weapon add name:Gunlance atk:wis def:dex|con`	*(add one, with its rules)*
`/weapon stats name:Gunlance atk:wis`	*(set or change them later)*
`/weapon stats name:Gunlance atk:any`	*(lift a restriction)*
`/weapon list`	*(see them all and what they allow)*
`/weapon remove name:Gunlance`	*(take one off)*

A weapon can dictate **which stats it fights with**, separately for attack and defence. A gunlance is fired rather than swung, so it might attack with **WIS** and defend with **DEX** or **CON**. Give several with **|** — **atk:str|dex** — and use **any** to lift a restriction.

A player carrying that weapon is then held to it: attacking with a stat it does not allow is refused, and the refusal names what they *can* use. **The auto-pilot obeys the same rules** — an NPC fighting on automatic picks the best stat its weapons permit rather than always reaching for STR.

NPCs carry weapons too. Give one on **/npc create** and the auto-pilot is bound by exactly the same rules — an NPC with a gunlance fires with WIS instead of reaching for its best raw stat. The slot only accepts weapons already on the server list, so an NPC can never hold something whose rules are unknown; **none** clears it, and omitting it leaves whatever they carry alone.

`/npc create name:Vault Warden str:6 con:5 \ weapon1:Gunlance`	*(an armed NPC)*
`/npc create name:Vault Warden atk_stat:Dexterity`	*(and how it should fight on auto)*
`/npc sheet name:Vault Warden`	*(shows what it carries and what that allows)*

Choosing How Auto Fights

Left alone, the auto-pilot reaches for whichever stat is **highest**. A character who fights with finesse over force can say otherwise — and so can a Game Master setting up an NPC.

`/char prefer atk:Dexterity def:Constitution`	*(your own, for when auto rolls for you)*
`/char prefer`	*(see what is set)*
`/char prefer atk:Clear it`	*(go back to using your best)*

This matters to players as well as Game Masters: **/fight auto** in full mode rolls for **everyone** in the fight, so without a preference your character swings with their strongest arm whether that suits them or not.

*A preference **never overrides a weapon**. If what you asked for is not one of the stats your weapons allow, your best allowed stat is used instead and **/char prefer** tells you so — setting one can only ever narrow a free choice, never break a rule.*

*A weapon with nothing set restricts nothing, so a list that predates this carries on unchanged. Carrying **one** unrestricted weapon frees the hand entirely; carrying two restricted ones lets you use either weapon's stats. An unarmed NPC is unrestricted, exactly as before.*

Derived Stats

Max HP is **CON plus a flat base**, 2 by default — so CON 10 gives 12 HP. A Game Master can change the base for the whole server with **/config hpbase base:3**, making it CON+3; set it to 0 for max HP equal to CON alone. Everyone's ceiling moves the moment it is changed, players and NPCs alike, though current HP is left where it is — run a rest or **hpfull @user** to top people up.

Stat	Formula	On change
Max HP	CON + base	HP always maxes

Stat	Formula	On change
Max Rerolls	LCK	Rerolls always max
Heal tracker	White Knight + WIS ≥ 5	Appears / disappears automatically

NPC System

All commands in this chapter are Game Master-only.

NPC Management

`/npc create name:Cave Orc str:14 con:10 dex:4 wis:2 lck:1 order:Black Knight`	*(full stat block)*
`/npc create name:Mystery Figure`	*(name only — stats added later)*
`/npc create name:Mystery Figure str:14 con:10`	*(fill stats in over time)*
`/npc copy name:Goblin new_name:Goblin 2`	*(duplicate an NPC, fresh HP)*
`/npc show name:Goblin`	*(full stat block for one NPC)*
`/npc hero name:Goblin stat:str`	*(make an NPC a Hero with a signature stat)*
`/npc hero name:Goblin remove:true`	*(strip Hero status)*
`/npc list`	*(shows stats and current HP)*
`/npc list category:Bandits`	*(only one category's NPCs)*
`/npc delete name:Aldric Vane`	

NPC HP & Healing

`/npc hp name:Goblin value:3`	*(set exact HP)*
`/npc hp name:Goblin`	*(omit value — full heal)*
`/npc heal names:all`	*(fully heal every NPC)*
`/npc heal names:Goblin, Orc`	*(fully heal the listed NPCs)*

Restoring Resources (/gmheal)

`/gmheal user:@a`	*(full HP — the default)*
`/gmheal user:@a restore:Everything`	*(HP, rerolls and heal charges)*
`/gmheal user:@a amount:Half`	*(restore half of maximum)*
`/gmheal user:@a amount:Add value:3`	*(add 3, capped at max)*
`/gmheal user:@a amount:Exact value:1`	*(set to an exact figure)*
`/gmheal npc:Goblin`	*(one NPC)*
`/gmheal npc:all`	*(every NPC at once)*

One command for every restore. Works on a player or an NPC (or **all** NPCs), and HP changes sync straight into any active fight. NPCs only have HP; heal charges are skipped for anyone who isn't a White Knight with WIS 5+.

global: restores everyone at once instead of naming a target — **Players**, **NPCs**, or **Everyone** together. It takes the same **amount** and **restore** options as a single target, so **/gmheal global:Everyone amount:Half** puts the whole server on half HP, and **/gmheal global:Players restore:Everything** hands every character HP, rerolls and heal charges

back. Pick exactly one of **user**, **npc** or **global**.

*Unlike scheduled recovery, a global heal does **not** skip players out on a quest. A Game Master typing this has decided to heal the room, and a silent exclusion mid-session would be a nasty surprise. Heal charges still only reach White Knights with WIS 5+, and every change syncs into any fight already running.*

*Stats are optional — an NPC can be registered by name (and given an avatar) with no stats at all, then statted up later. Re-running **/npc create** with the same name updates only the fields you supply and leaves the rest untouched.*

NPC HP persists between fights. Knocked-down NPCs (0 HP or less) are left out of new fights until restored.

Speaking as an NPC

```
/pr say name:Cave Orc speech:Halt! Who goes there?      (speech — in quote marks)
/pr say name:Cave Orc action:raises its axe             (action — italic emote)
/pr say name:Cave Orc action:raises its axe             (both, stacked)
  speech:Halt!
/pr say name:Cave Orc                                   (opens a writing box)
```

***Works in threads and forum posts too.** A thread cannot own a webhook, so the NPC's voice is created on the parent channel instead and every message is routed back into the thread it was called from — several threads under one channel share the same NPC webhook.*

Posts as the NPC through their webhook — their name and avatar, no dice rolled. Fill **action**, **speech**, or both: the action is italicised and the speech is wrapped in quote marks, stacked on separate lines. Leave both blank and a **writing box** opens with roomy multi-line fields — easier for longer roleplay. Use **raw** to post exactly as typed.

Rolling as an NPC (/pr shorthand)

```
/pr roll name:Cave Orc notation:1d20+8 label:strike
  flavour:The orc lunges
/pr create name:Aldric Vane str:8 con:6 dex:10 wis:4
lck:2
/pr list
```

Posts via webhook — appears as the NPC with their name and avatar.

Setting NPC Avatars

1. Admin runs **/config npcchannel #channel** to set the image bank channel.
2. Game Master uploads an image to that channel with the NPC name as the message text.
3. Bot adds a checkmark reaction to confirm. 4. Re-upload with the same name to update.

Bot requires Manage Webhooks permission in the server.

Heroes & Signature Stats

Hero is a Game Master-granted class, not a player choice — players who try to set it are refused. Both player sheets and NPCs can be Heroes.

```
/char set field:class value:Hero user:@a        (grant Hero to a player sheet)
/char signature user:@a stat:str                (designate their signature stat)
```

`/npc hero name:Aldric stat:dex`	*(Hero NPC with a signature stat)*

A Hero may have one **signature stat** with **5 or more** points. Every roll using that stat — typed shorthand, **/roll**, and both attack and defence in fights, manual or automatic — is made with **advantage**, marked on the roll card. If the stat later drops below 5 the advantage simply stops applying until it is restored; an explicitly requested disadvantage still wins.

Fight System

Structured fights with initiative, turn order, stat-based rolls and damage tracking.

Commands

`/fight addnpc npc:Goblin, Orc`	*(add NPC(s) mid-fight, Game Master)*
`/fight order sequence:@a, Goblin, @b`	*(reorder players + NPCs, Game Master)*
`/fight atk stat:str npc:Goblin target:@user`	*(Game Master attacks AS the NPC)*
`/fight def stat:dex npc:Goblin`	*(Game Master defends AS the NPC)*
`/fight skip`	*(skip the current turn — fighter stays in, Game Master)*
`/fight hp value:3 target_npc:Orc`	*(set HP mid-fight, sheet synced, Game Master)*
`/fight kick target_npc:Orc`	*(remove a fighter, fight continues, Game Master)*
`/fight refill npcs:all`	*("all" or names — refill reroll tokens, Game Master)*
`/fight auto mode:Full ...`	*(bot resolves whole fight, Game Master)*
`/fight auto mode:Full teams:@a @b vs Goblin, Orc`	*(party-vs-monsters sides, Game Master)*
`/fight auto mode:NPCs only ...`	*(bot plays NPCs, Game Master)*
`/fight auto mode:Demo`	*(example showcase, Game Master)*
`/fight auto ... practice:true`	*(a bout in any auto mode, Game Master)*
`/fight end`	*(Game Master only)*

How a Fight Runs

When a fight starts, the bot rolls DEX initiative (1d20 + DEX) for every fighter and orders the turn order from highest to lowest, ties broken by the raw d20. Add **manual:true** to skip the roll and keep your listed order, or use **/fight order** with a comma-separated sequence.

Knocked-down fighters: anyone at 0 HP or less is left out when a fight starts or when NPCs are added, with a warning naming them. Restore NPCs with **/npc heal** or **/npc hp**, players with rests or **hpfull @user**.

Practice bouts: add **practice:true** to **/fight start** or **/fight auto** and the fight becomes a friendly spar. Everything runs exactly as normal — initiative, rolls, damage, crits, carry-over effects, rerolls, recap — but the cut-off moves from 0 HP to **2 HP**. A fighter bows out the moment they reach 2, and damage never carries anyone below it, so nobody leaves the yard worse than winded. Sparring partners already at 2 HP or less are left out at the start, the same way knocked-down fighters are. The bout is announced on start, marked on **/fight status**, and noted again in **/fight log**, so a spar can never be mistaken for the real thing.

HP stays in sync: any mid-fight HP change — **!hp**, **!heal**, rests, **/npc hp** or the **/fight hp** command — is mirrored straight into the fight, so a heal is never overwritten by the next exchange. Anywhere a command takes NPC names, **category:Name** adds a whole category at once.

When a Fight Ends

However a fight finishes — a knockout, a forfeit, a kick, **/fight end** or an auto-resolve — a single public **result post** goes to the channel where everyone can read it. Nothing important is left in a private reply: the Game Master's confirmation for **/fight end** stays private, but the result itself is posted for the whole table.

The post names the **victor**, then lists every combatant's **final standing** — exact HP for players, a condition band for NPCs while their stats are hidden — followed by the recap.

The **recap** covers players and NPCs alike, in three parts. **Rolls**: how many attacks and defences each fighter made, their average total, and their best and worst natural dice. **Damage**: dealt and taken, natural 20s and 1s, and rerolls spent. **Blow by blow**: every exchange in order — both rolls, the natural die and the final total for each side, and whether it landed or was blocked, with natural 20s and 1s marked. Very long fights show the most recent exchanges and say how many earlier ones were trimmed; the roll and damage figures still cover the whole fight. Long recaps are split across messages so nothing is lost to Discord's length limit.

/fight log re-posts the last finished fight's recap in that channel at any time, blow-by-blow included.

NPCs as combatants: a Game Master lists NPCs in **npcs:** (comma-separated) on start or **/fight addnpc** later. On an NPC's turn the Game Master adds **npc:Name** to **/fight atk** or **/fight def**; attack an NPC with **target_npc:Name**.

Auto Modes

/fight auto mode:Full rolls attack, defend and damage for everyone and plays the fight through to a winner, saving HP to each combatant's sheet as it changes. Add **teams:@a @b vs Goblin, Orc** for proper sides — fighters only target the enemy team, and the last team standing wins. **mode:NPCs only** starts a normal fight where the bot takes the NPCs' turns automatically while players still use **/fight atk** and **/fight def**. **mode:Demo** runs a throwaway example.

Automatic rolls always use the fighter's highest of STR or DEX, for attack and defence alike (ties go to STR), and post the same full roll card as a manual roll. Each NPC's cards appear under that NPC's own name and avatar (via webhook).

NPC rerolls: in auto modes each NPC carries its own reroll tokens — LCK per fight. A token is spent only when the natural die shows **8 or less** (server-tunable via **/config npcreroll**; 0 disables): a defender about to be hit rerolls first, then a blocked attacker may answer — one each per exchange. **/fight status** shows remaining tokens, and **/fight refill** restores them mid-fight.

Damage

Situation	Damage
Attack total ≥ Defence total	1
Attacker natural 20	+1 bonus
Defender natural 1	+1 bonus
Attacker nat 20 + Defender nat 1	4 total
Defence total > Attack total	0 (blocked)
Fighter reaches 0 HP	Knocked down, removed from turn order, HP goes negative

Critical Effects (carry-over)

A natural 1 or natural 20 leaves a mark on the *next* roll, in both manual and auto fights:

Trigger	Effect on the next roll
Natural 1 on an attack	The attacker fumbles — their next defence is rolled as a flat d20 (no stat, no advantage).
Natural 20 on a defence	The defence turns the blow aside and the defender gains +2 on their next attack — unless that attack was itself a natural 20.

Each effect is consumed by the affected fighter's next matching roll and is announced when it lands ("presses the riposte", "defends on a flat d20"). Leaving a fight clears any pending effects.

Merits & Ranks

A milestone-style progression system. Merits are a lifetime tally a Game Master awards; ranks are named tiers with merit thresholds. The bot tracks progress and flags eligibility, but every promotion is decided by a Game Master.

Merits

`/merit add user:@player amount:2`	*(award merits — Game Master)*
`/merit remove user:@player amount:1`	*(take merits away — Game Master)*
`/merit set user:@player amount:10`	*(set an exact total — Game Master)*

Ranks

`/rank add name:Knight threshold:5`	*(create or update a rank — Game Master)*
`/rank add name:Squire threshold:0 order:0`	*(order sets junior→senior — Game Master)*
`/rank promote user:@player rank:Knight`	*(set a player's rank — Game Master)*
`/rank eligible`	*(who has met a threshold but isn't promoted — Game Master)*
`/rank remove name:Squire`	*(delete a rank — Game Master)*

/merit view *shows current merits and exactly how many more are needed for the next rank, e.g. "Knight · 7 merits · 8 to Paladin". Every merit change is recorded — quest rewards show the quest name, manual changes show "by GM" — so* ***/merit history*** *answers "who earned what, and when". Removing a rank doesn't change players who already hold its label.*

Activities

An activity is a minigame you write for your server: the bot narrates, asks for rolls, branches on the results and loops until someone stops. Fishing, foraging, a gauntlet in the training yard — whatever you script. Only a Game Master can write one; whether players can **start** one is a setting.

Writing One

Paste a script into any channel the bot can read, starting with **[ACTIVITY] Name**. Re-pasting the same name replaces it. The whole script is checked before anything is saved.

```
[ACTIVITY] Fishing
TALLY renown                          (a running total, paid out at the end)

SCENE find
SAY Find a spot to set up.
ROLL wis DC15                         (a difficulty to beat)
  PASS -> cast
  FAIL ONE OF                         (picks a different line each loop)
    This spot does not look all too lucky...
    They are not biting here today...
  FAIL -> find                        (loops back)

SCENE cast
ROLL str|dex|wis                      (the roller picks the stat)
  1-5   Small fry.  -> fight_small    (ranges on the total)
  16+   A monster!  -> fight_extra

SCENE fight_big
GAUNTLET str|con 14 12 10             (three rolls, each harder to fail)
  NAT20 It leaps aboard. -> caught
  NAT1  Your line snaps. -> restring
  PASS -> caught
  FAIL -> find

SCENE caught
GAIN renown 3                         (banked on arrival)
CHOICE                                (buttons, no dice)
  Carry on      -> cast
```

```
  Call it quits -> depot

SCENE depot
END TALLY                                   (pays the tally out as renown)
```

What Each Line Does

Line	Meaning
SCENE name	A step. The first one is where a run begins.
SAY ...	Narration. Runs over as many lines as you like.
AS Cave Orc	Speak this scene in an NPC’s voice, through their webhook.
ROLL str	Ask for a roll. str|dex|wis lets the roller choose.
ROLL wis DC15	Beat 15 for PASS, else FAIL.
GAUNTLET str|con 14 12 10	A run of rolls, each with its own DC. All must pass.
GAUNTLET 14:str 12:str|con 10:dex	The same, but a different check at every step.
1-5 text -> scene	Branch on the roll total. 16+ is open-ended.
PASS / FAIL text -> scene	Branch on the outcome band.
BAND ONE OF	Indented lines below become random variants.
NAT20 / NAT1 text -> scene	Overrides everything else.
CHOICE	Buttons instead of dice. Options are label -> scene.
TALLY renown	Names a running total, declared once at the top.
GAIN renown 3	Adds to the tally when a player arrives at this scene.
END	Finishes. END TALLY pays out, merits:2 awards merits,
	rewards:a silver key is announced for you to hand out.

Without a **DC** or ranges, outcomes fall back to the same bands a **?** check uses: **CRIT** a natural 20, **PASS** 15+, **PARTIAL** 10–14, **FAIL** under 10, **FUMBLE** a natural 1. You need not define all of them — a crit falls back to PASS, a fumble to FAIL.

*Validation refuses a branch pointing at a scene that does not exist, a duplicate scene name, a scene with no roll, choice or ending, a roll on something that is not a stat, a gauntlet longer than eight, and a **GAIN** with no **TALLY** — each with the reason, so a run can never dead-end.*

Running One

`/activity demo`	*(play the built-in fishing game (Game Master))*
`/activity run name:Fishing`	*(start it in this channel)*
`/activity list` `/activity show name:Fishing`	*(what exists, and read it back in full)*
`/activity stop`	*(abandon the run here)*
`/activity set name:X scene:find field:Roll value:dex`	*(tweak one line (Game Master))*
`/activity delete name:X`	*(remove it (Game Master, asks first))*
`/config activities players:true`	*(let players start them too)*

/activity demo plays a ready-made fishing game so you can see the whole system working before writing anything: a difficulty check that loops with a different excuse each time, four sizes of catch chosen by the roll, a gauntlet per size, natural 20s and 1s, and buttons to keep going or head back. It awards **nothing** — no renown, no merits, no items — so it is safe to play with.

One Run Each

An activity run belongs to the person who started it. Several people can play in the same channel at once, each with their own prompts, their own progress and their own rewards — and one person stopping or wandering off does nothing to anyone else. Every post is tagged with whose run it is, and a button only answers for its owner.

`/activity run name:Fishing`	*(start your own run)*
`/activity stop`	*(end yours — a Game Master with none running can clear the channel)*

Answering a Scene

Press the button, or **type the stat and add your own flavour after it** — both roll the same thing, but typing lets you say what your character is doing.

`wis I survey the reeds where the current slows`	*(rolls WIS, prints your words)*

A typed roll only answers the scene if the stat is one that step accepts; anything else falls through to an ordinary roll, untouched. Your stats are shown alongside the result either way.

A Second Chance

After a roll the tale **waits** rather than moving straight on. If you have a reroll to spend you are offered one, alongside a **Carry on** button:

`↻4 Reroll (2 left)`	*(spend one and roll again)*
`▶ Carry on`	*(take the result and continue)*

Only one reroll is offered per roll — the second result stands, and the tale moves on. If you have none left, or the scene was an ending, it continues as before without stopping to ask. Your progress is saved while it waits, so nothing is lost if you take a moment to decide.

*In **/activity demo** the reroll is **free** and always offered, even with none left — a dry run should not quietly cost a player their real rerolls just to see what the button does.*

Each scene posts with a button per stat it accepts. **Anyone in the channel can press one** — the roll uses their own sheet, honours a Hero's signature stat, and lands in the roll audit like any other. One run per channel at a time. Writing and deleting always need a Game Master; starting one is GM-only until **/config activities players:true**.

Renown

Renown is not a currency. It is a running tally of how a character **stands in the world** — what they have done, who has noticed, and how far their name carries. It is earned from quests, encounters and activities, and it is not meant to be traded away for goods. Someone with high renown is **known**; that is the whole of it.

A Game Master can adjust it either way when the story calls for it — a reputation can be damaged as well as built — but it is a record of standing rather than a purse.

`/renown view` `/renown view user:@player`	
`/renown leaderboard`	*(who is best known)*
`/renown history`	*(where a standing came from)*
`/renown gain user:@a amount:5 reason:Cleared the Sunken Vault`	*(Game Master)*
`/renown loss user:@a amount:3 reason:Disgraced at court`	*(Game Master)*
`/renown set user:@a amount:0`	*(Game Master)*

Every change is logged with its reason, so **/renown history** answers how a reputation was built and where it was lost.

Merit

Merit is the earned measure of service — awarded by a Game Master, accumulated across quests and activities, and the thing rank thresholds are set against. Unlike renown, **merit is tradeable**: it can be passed between players, and potentially to and from NPCs, as payment, tribute, a debt settled or a favour bought.

`/merit give user:@a amount:2 reason:A debt settled`	*(offer some of your merit)*
`/merit trades`	*(what is still waiting on a Game Master)*
`/merit cancel id:3`	*(withdraw one — your own, or any as a Game Master)*

No merit moves until a Game Master signs it off. An offer is held and posted to the sheet approval channel with Approve / Refuse buttons; a refusal asks the Game Master why and passes the reason back. Both parties are told when it lands, and it is announced in the channel where it was offered so the table sees the trade happen.

Your balance is checked twice — when you offer, and again when a Game Master approves. If you have spent the merit in between, the trade is voided rather than pushing anyone into the red.

Renown says who you are in the world. Merit is what you have earned and may hand on. A character can be widely known and hold no merit at all, or quietly hold a great deal.

NPC Records

An NPC keeps the same records a player does. **/npc sheet** shows the lot on one page — stats and HP, standing, inventory, lifetime roll history and lore.

`/npc sheet name:Cave Orc`	`/pr sheet name:...`	*(the whole record)*
`/npc give name:Cave Orc item:...`	`/pr give name:...`	*(hand them something)*
`/npc take name:Cave Orc id:1`	`/pr take name:...`	*(take it back)*
`/npc npclore name:Cave Orc text:...`	`/pr npclore`	*(write their story)*
`/npc delete name:Cave Orc`	`/pr delete name:...`	*(remove them entirely)*

Their dice count too. Every roll the auto-pilot makes for an NPC — attacks, defences, reroll answers, initiative — goes into that NPC's lifetime tally, so a long-running villain builds a record of their own luck exactly as a player does.

Merit and renown work on an NPC the same way they do on a character, so an NPC can hold standing in the world, be paid in merit, or carry the reward for a job.

Categories

`/npc categorycreate name:Bandits`	*(make a grouping)*
`/npc categoryassign name:Cave Orc category:Bandits`	*(file an NPC under it)*
`/npc categoryremove name:Cave Orc`	*(take it out)*
`/npc categorylist`	*(every category and who is in it)*
`/npc categorydelete name:Bandits`	*(remove the grouping)*

*Everything above also works on **/pr**, which is the same command under a shorter name for use mid-scene.*

The Fallen

When a character is brought to 0 HP they are down, not gone — whether that is the end is the Game Master's call. **/gmkill** makes it final and writes them up.

`/config memorial channel:#gm-records public:#the-fallen`	*(both halves)*
`/gmkill user:@player`	*(call it, once they are down)*
`/gmkill npc:Cave Orc`	*(NPCs too)*
`/gmkill user:@player anyway:true`	*(even while still standing)*
`/gmrevive user:@player`	*(bring them back)*

It asks for a **cause of death**, optional **last words** and an optional **epitaph**. Everything else is already known: their name, order, rank at the moment they fell, merit, renown — and their **deeds**, which are not typed out but gathered from what the bot watched them do. Quests seen through and how long each took, the merit they earned and what for, the standing they won, the dice of a lifetime, and what they were carrying at the end.

It is written up **twice**. The **GM record** carries the full account — merit, renown, the dice of a lifetime, what they carried, when they fell — with a **Revive** button attached. The **public hall** gets the same life told plainly: name, order, rank, cause, deeds, last words and epitaph, and **no figures at all**. No merit or renown totals, no roll history, no inventory, no dates hanging off the deeds. A hall should hear what someone did, not what they were worth.

Pressing **Revive** brings them back at full HP and **deletes both posts** — a death that was undone does not linger in the hall. If the same character falls again, the button carries a tally beside it: **Fallen 3× · revived 2×**.

Nothing is deleted from the character. A fallen one keeps their whole record, so a revival costs nothing and the account can be rebuilt. Until they are brought back they cannot roll: the fallen take no more actions.

Publishing These Books

The three books are built outside the bot and committed to a repository. Point the bot at that repository and it will keep **one current post** of all three in a channel of your choosing — when a new build lands, the old post is deleted and replaced rather than the channel filling up with stale copies.

`/config docs channel:#gm-books repo:owner/name`	*(set it up)*
`/config docs player_channel:#rules`	*(player book alone, posted silently)*
`/config docs branch:live path:docs`	*(if not on main, or not at the root)*
`/config docs push:true`	*(fetch and repost right now)*
`/config docs`	*(what is being watched, and the current post)*
`/config docs disable:true`	*(stop watching)*

It looks every **15 minutes**, and **pings the Game Master roles** whenever it publishes. Checking is cheap — it asks GitHub for the file versions rather than downloading, so a check that finds nothing new costs three small requests.

Give it a **player_channel** as well and the **player book alone** is kept current there — posted **silently**, with no role pinged and no notification raised, so a reference channel stays up to date without nagging anyone. It replaces its own previous post the same way.

*The new post goes up **before** the old one comes down, so a failure part-way leaves the channel with a copy rather than none at all. The repository must be public, and the three files must be named **DDice-Commands-GameMaster.pdf**, **DDice-Commands-Player.pdf** and **DDice-Commands-Parchment.pdf**.*

Test Tools

Trying a feature out usually means inventing a quest or an NPC you then have to tidy out of the world. **/gmtest** makes throwaway ones instead. Everything it creates is named **[test]** and can be swept away in one command. It is hidden from players entirely.

`/gmtest quest`	*(a quest with you on the party, in this channel)*
`/gmtest npc`	*(an NPC with items, standing, rolls and lore already on it)*
`/gmtest list`	*(what it has made)*
`/gmtest clean`	*(delete all of it — asks first)*

The test quest arrives ready to start, so the clock, the reminders, the timeline and the summary can all be exercised in a few minutes. The test NPC arrives with a record already on it, so **/npc sheet** has something to show.

*Cleaning only ever touches rows named **[test]**, and clears their events, summaries, inventory, roll tallies and standing log along with them.*

Quest Board

Running a Quest — the Clock

Starting a quest with **/quest start** begins a stopwatch. From then on the bot posts a public time check in the quest's run channel **every 15 minutes**, and **on the hour** a recap of everything that happened during it. Set the channel first with **/quest runchannel**, or there is nowhere for them to go.

`/quest start number:1`	*(the clock begins)*
`/quest note number:1 text:They bribed the gatekeeper kind:Roleplay`	*(mark a moment)*
`/quest timeline number:1`	*(the whole log so far)*
`/quest pause number:1`	*(stop the clock, keep the time)*
`/quest resume number:1`	*(carry on where it left off)*
`/quest complete number:1`	*(stop, award, and write it up)*

A timeline reads back like a ship's log:

```
` 0h 00m` ⚑ Quest begins — 4 on the party
` 0h 15m` □ Time check — 0h 15m
` 0h 17m`ˆId Cave Orc speaks
` 0h 44m` ⚔ Artorius wins the fight
` 1h 00m` □b Hourly recap — 3 events
```

Combat and roleplay log themselves. A fight ending in the quest's run channel, or an NPC speaking there through **/pr say**, is attached to whichever quest is running in that channel. Anything else you want on the record goes on with **/quest note**, taggable as roleplay, combat or a plain note.

Pause keeps everything. *The time already run is banked and the clock stops — a paused quest gets no reminders and logs nothing. Resuming picks up at exactly the same figure. Both counters live on the quest itself, so a restart or redeploy mid-session resumes rather than starting the count again.*

The Quest Summary

On **/quest complete** the clock stops and the whole run is written up: who ran it, how they run a table, how long it took, who was on the party, and the full timeline. Set where it goes with **/config questlog channel:#chronicle**.

`/config questlog channel:#chronicle`	*(where finished quests are written up)*
`/config questlog disable:true`	*(stop posting them)*

The summary is then **linked on every party member's standing page — /char standing** and **/char summary** both list the quests a character has finished, each one a link straight to its write-up, with how long it took and how long ago it was.

Several GMs, the Same Adventure

A quest holds one party on one clock, so two Game Masters cannot share a quest number. **/quest instance** makes a separate run of the same adventure: it copies the writing — name, lore, objectives, details, rewards, merit and party rules — and leaves everything else fresh. A new number, an empty party, a clock at zero and its own log.

```
/quest instance number:1                                  (your own run of quest 1)
/quest instance number:1 gm_style:Roleplay-focused        (and advertise how you run it)
```

Three Game Masters can run the same adventure at once, each with their own party, channel, clock and summary. Completing one does nothing to the others. The board marks them: One of 3 separate runs of this adventure.

GM Style

A quest can advertise how its Game Master runs a table, so a player knows what they are applying to. Set it on **/quest create** or per instance; it shows on the quest card, the board post and the final summary.

Tag	What it promises
□ Mechanics-focused	rules, rolls and tactics to the fore
˜Id Roleplay-focused	character and conversation to the fore
□ Mixed elements	a bit of both
□ Combat-heavy	expect fighting
˜Ḥ9 Puzzle & investigation	problems to work out
˜Ya Sandbox	the players decide where it goes

GMs create quests with lore, objectives, details and rewards. Quests are posted as a message with an Apply button and also appear on the board. Players apply, a Game Master approves the party, and on completion merits are auto-awarded while other rewards are listed for the Game Master to hand out.

For the Game Master

`/quest create name:Goblin Cave objectives:... merit_reward:2 party_size:4 hard_cap:true`	*(create a quest)*
`/quest post number:1 channel:#board`	*(post as an embed with an Apply button)*
`/quest approve number:1 user:@a force:true`	*(approve an applicant · force past a hard cap)*
`/quest kick number:1 user:@a`	*(remove a member or applicant)*
`/quest runchannel number:1 channel:#thread`	*(set where it runs & rewards)*
`/quest start number:1`	*(lock the party, mark in progress)*
`/quest complete number:1`	*(finish — auto-award merits, list rewards)*
`/quest delete number:1`	*(remove a quest permanently)*

Quests are auto-numbered for easy recall, e.g. ***#001-Goblin Cave*** *(repeatable quests keep the name and get a fresh number each time). Party size is a hard cap or a suggestion — the Game Master chooses with* ***hard_cap****, and* ***force:true*** *on approve overrides a cap. Point a quest at a thread with* ***/quest runchannel*** *and its completion rewards are announced there.*

Penned by the DDice Scriptorium · May your rolls be ever in your favour